Demo app: PharmacyApi — .NET 8 Web API (C#)

Infrastructure as Code & Secure Pipelines (DevSecOps)

Provisioning, Securing and Scanning Your Delivery Pipeline

What We'll Cover

1. Why manual provisioning breaks down, and what Infrastructure as Code fixes
2. Bicep and Terraform: syntax, state, and how to choose between them
3. Provisioning environments repeatably from a pipeline
4. Secrets management with Azure Key Vault
5. Managed identities and least-privilege service connections
6. Secret, dependency and container scanning as pipeline gates
7. Policy and compliance as code
8. Putting it all together into one secure pipeline

Learning Objectives

- Explain what Infrastructure as Code is and why it replaces manual, click-driven provisioning
- Write and read a basic Bicep or Terraform resource definition for an Azure App Service
- Store and retrieve application secrets from Azure Key Vault without hard-coding them anywhere
- Configure a managed identity and a least-privilege service connection for a pipeline
- Add secret, dependency and container scanning steps that block a build on a critical finding
- Describe how Azure Policy enforces compliance automatically at deployment time
- Trace a single change through a full provision → secure → scan → deploy pipeline

PART 1

Infrastructure as Code Fundamentals

The Problem With Manual Provisioning

- An environment built by clicking through the Azure Portal exists only in one place — nobody can see what was actually configured without logging in and looking
- It can't be reviewed before it's created — there's no pull request for a sequence of portal clicks
- It can't be recreated reliably — ask two engineers to rebuild the same environment by hand and you'll get two different results
- A disaster recovery plan that says “recreate the resource group” is not a plan if nobody wrote down exactly what was in it

A familiar scenario

A production App Service has a setting nobody remembers configuring. It works, so nobody touches it — and nobody can explain it either. That's the cost of undocumented infrastructure.

What Is Infrastructure as Code

- Infrastructure — resource groups, App Services, Key Vaults, networking, databases — is described in a file, not clicked together one setting at a time
- That file lives in source control: it's reviewed in a pull request, diffed like any other change, and versioned alongside the application it supports
- A tool reads the file and reconciles the real environment to match it — creating, updating or removing resources as needed
- Running the same file twice produces the same environment — repeatability is the entire point, not a side benefit

In one sentence

If you can't tell what your environment looks like by reading a file in your repository, it isn't Infrastructure as Code yet — it's still a guess.

Declarative vs. Imperative Provisioning

Declarative (Bicep, Terraform)

- You describe the end state you want
- The tool works out the steps to get there
- Running it again is safe — it only changes what's different
- This is the model both Bicep and Terraform use

Imperative (a script of commands)

- You write the exact sequence of steps to run
- You're responsible for every step being correct and in order
- Running it twice can fail, or duplicate resources, unless you plan for that
- A PowerShell or Azure CLI script that creates resources is imperative

Why It Matters: Repeatability, Review, Recovery

- Repeatability — Dev, Staging and Production can all come from the same template, with only the parameters changing, so they stay genuinely consistent
- Review — a change to the environment goes through the same pull-request process as a change to the application, with a reviewer who can see exactly what will happen
- Recovery — if an environment is accidentally deleted or corrupted, the template rebuilds it from scratch, instead of someone trying to remember what used to be there
- Audit trail — git history shows who changed the infrastructure, when, and why, the same way it does for application code
- Cost control — a reviewer can see a new, expensive resource being added before it's created, not after the first bill arrives

The Infrastructure as Code Landscape

- ARM templates — Azure's original JSON-based format; verbose but still the format everything else compiles down to
- Bicep — Microsoft's own domain-specific language that compiles to ARM JSON; concise syntax, tightest fit for Azure-only environments
- Terraform — HashiCorp's HCL language; works across Azure, AWS, GCP and many other providers through one consistent workflow
- Pulumi — lets you write infrastructure definitions in a general-purpose language (including C#); useful when a team wants full programming constructs

This module's focus

We'll go deep on Bicep and Terraform — the two most common choices for an Azure-centered .NET shop — and compare them directly.

Bicep: What It Is and How It Works

- A domain-specific language written specifically for describing Azure resources, authored by the Azure team
- Every .bicep file compiles to ARM JSON behind the scenes — Bicep is a cleaner syntax layered over the same deployment engine
- No separate state file to manage — Azure itself is always the source of truth for what currently exists
- Deployed with the Azure CLI (az deployment group create) or from an Azure Pipelines task
- Strong editor support in Visual Studio and VS Code, including IntelliSense and resource schema validation

Best fit

Teams that are Azure-only and want the tightest possible integration with the platform, with nothing extra to install or configure.

Code: Bicep App Service Definition

- This defines the App Service plan and web app that will host the PharmacyApi .NET 8 Web API, parameterized by environment

```
param environmentName string
param location string = resourceGroup().location

resource appPlan 'Microsoft.Web/serverfarms@2023-01-01' = {
  name: 'asp-pharmacyapi-${environmentName}'
  location: location
  sku: { name: 'P1v3', tier: 'PremiumV3' }
}

resource webApp 'Microsoft.Web/sites@2023-01-01' = {
  name: 'app-pharmacyapi-${environmentName}'
  location: location
  properties: {
    serverFarmId: appPlan.id
    siteConfig: { netFrameworkVersion: 'v8.0' }
  }
}
```

Terraform: What It Is and How It Works

- Written in HashiCorp Configuration Language (HCL), designed to work consistently across many cloud providers
- The azurerm provider translates HCL resource blocks into calls against the Azure Resource Manager API
- Terraform keeps its own state file recording what it believes currently exists — every plan and apply compares the template against that state
- Deployed with terraform init, terraform plan and terraform apply, often wired into a pipeline stage
- A much larger ecosystem of providers beyond infrastructure — DNS, monitoring, SaaS platforms, and more, all through the same workflow

Best fit

Teams that are multi-cloud, or that want one consistent tool and workflow across every platform they provision.

Code: Terraform App Service Definition

- The same PharmacyApi hosting environment, expressed in Terraform's HCL syntax against the azurerm provider

```
resource "azurerm_service_plan" "pharmacyapi" {
  name                = "asp-pharmacyapi-${var.environment_name}"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  os_type             = "Linux"
  sku_name            = "P1v3"
}

resource "azurerm_linux_web_app" "pharmacyapi" {
  name                = "app-pharmacyapi-${var.environment_name}"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  service_plan_id     = azurerm_service_plan.pharmacyapi.id
  site_config { application_stack { dotnet_version = "8.0" } }
}
```

Bicep vs. Terraform: Choosing Between Them

Bicep

- Azure-native, compiles to ARM JSON
- No separate state file to store or protect
- Tightest fit with new Azure resource types
- Simplest choice when you're Azure-only

Terraform

- Works across Azure, AWS, GCP and more
- Keeps its own state file — store it remotely and protect it
- Larger ecosystem beyond cloud infrastructure
- Simplest choice when you're multi-cloud

Pick one for your organization, not both — mixing tools against the same resources is how state and reality quietly drift apart.

State Management & the Terraform State File

- Terraform's state file is a JSON record of every resource it manages and the values it last saw for each one
- Every plan compares three things: your template, the state file, and the real infrastructure — then shows you the difference
- The state file must be stored somewhere shared and durable — typically an Azure Storage account configured as a remote backend
- Two engineers running terraform apply from their own laptops, against local state files, is a recipe for conflicting, silently overwritten changes
- Bicep has no equivalent file to manage — Azure Resource Manager itself plays that role, which simplifies this specific problem

A common mistake

Committing a local terraform.tfstate file to source control. It often contains sensitive values and causes state conflicts the moment a second person applies changes.

Idempotency, Plan and What-If

- Idempotent means running the same deployment twice produces the same result — the second run changes nothing because the environment already matches
- terraform plan shows exactly what would change before anything actually runs — resources to add, change or destroy, color-coded
- Bicep's equivalent is what-if: az deployment group create --what-if prints the same kind of preview without touching anything
- A pipeline should always review this output before applying — a plan that proposes deleting a production database is a signal to stop, not proceed

Why this matters

A destructive change hidden inside an otherwise routine deployment is exactly the kind of mistake a human reviewer can catch in a plan output — but never in a portal click.

Modularizing Infrastructure Code

- A single giant template becomes unreadable fast — both tools support breaking definitions into reusable modules
- Bicep modules: a module block references another .bicep file, passing in parameters — for example, a reusable App Service module reused across every environment
- Terraform modules: a folder of .tf files, invoked with a module block and its own inputs and outputs, often published and versioned independently
- Common modules worth building once: networking, App Service hosting, Key Vault with standard access policies, SQL Database with standard backup settings
- The payoff: a change to how App Services are configured updates every environment that uses the module, instead of being copy-pasted and drifting apart

Parameterizing Environments

- The same template provisions Dev, Staging and Production — only the parameter values change between them, never the template itself
- Bicep: separate .bicepparam files (dev.bicepparam, staging.bicepparam) feed the same main.bicep
- Terraform: separate .tfvars files (dev.tfvars, staging.tfvars) feed the same root module
- Typical parameters that vary: SKU/tier, instance count, whether diagnostic logging is verbose, which Key Vault to read secrets from
- What should never vary between environments: the resource types themselves — if Staging uses a different App Service plan tier, it's no longer validating what Production will actually run

Rule of thumb

If you can't name exactly which parameter file produced a given environment, parameterization isn't being used consistently yet.

Provisioning From the Pipeline

- A Provision stage runs before the application ever deploys — it ensures the target environment exists and matches the template before anything is pushed to it

```
- stage: Provision
  jobs:
  - job: ProvisionInfrastructure
    steps:
    - task: AzureCLI@2
      inputs:
        azureSubscription: 'arm-pharmacyapi-dev'
        scriptType: 'bash'
        inlineScript: |
          az deployment group create \
            --resource-group rg-pharmacyapi-dev \
            --template-file infra/main.bicep \
            --parameters infra/dev.bicepparam
```

Knowledge Check

A template is applied twice in a row against the same environment, with no changes made in between. What should happen the second time?

A. The deployment fails, because the resources already exist

B. Nothing changes — the tool compares the template to reality and finds no difference to apply

C. Every resource is deleted and recreated from scratch

D. The template silently skips execution and reports success without checking anything

Answer

B — Correct

Both Bicep and Terraform compare the desired state in the template against the real infrastructure. When nothing has changed, there is nothing to reconcile — that's what makes the tool safe to re-run.

- A is wrong — a correct declarative deployment never fails just because resources already exist in the expected state
- C is wrong — that would defeat the entire purpose of a declarative tool; only Terraform's explicit destroy or a forced recreate behaves that way
- D is wrong — the tool does check; it just finds the current state already matches, so there's nothing to change

PART 2

Secrets Management & Identity

The Problem With Hard-Coded Secrets

- A connection string, API key or certificate password typed directly into appsettings.json or a pipeline YAML variable is one accidental commit away from being public
- Removing it in a later commit doesn't help — git history keeps every version of every file forever, visible to anyone who can clone the repository
- A secret printed to a build log is just as exposed as one committed to source control — logs are often retained and shared more widely than people expect
- The only real fix once a secret has leaked is rotating the actual credential at its source — editing the file is not enough

A common mistake

Pasting a real database connection string into appsettings.Development.json “just for now.” It's in history the moment it's committed, whether or not the file is removed afterward.

Azure Key Vault: Overview

- A managed service that stores secrets, keys and certificates outside of application code, configuration files and the pipeline itself
- Applications and pipelines request a secret's value at run time through an authenticated call — the value is never stored anywhere else
- Every access is logged, so you can see exactly which identity read which secret and when
- Secrets can be versioned, rotated and have expiration dates set — an expired secret stops being usable automatically
- A vault's access is itself permissioned — granting one identity access to one vault does not grant access to every vault in the subscription

For PharmacyApi

The SQL connection string, any third-party API key, and the signing key for issued tokens all live in Key Vault — never in appsettings.json.

Access Policies vs. Azure RBAC for Key Vault

Vault access policies (legacy)

- Permissions set directly on the vault resource
- Granular per-operation control (get, list, set)
- Doesn't integrate with Azure's broader RBAC model
- Still common in older environments

Azure RBAC for Key Vault (recommended)

- Permissions managed the same way as any other Azure resource
- Roles like Key Vault Secrets User, Key Vault Secrets Officer
- Consistent with how every other resource is secured
- Microsoft's current recommended approach for new vaults

Linking Key Vault to Pipeline Variable Groups

- In Azure Pipelines, a variable group under Library can link directly to an Azure Key Vault instead of storing values itself
- Once linked, every secret in that vault becomes available as a pipeline variable — referenced as $(secretName) in YAML
- The actual secret value is never written into the pipeline definition, never shown in logs, and never stored in the pipeline's own variable store
- The service connection used for the link needs Key Vault read access — nothing more — scoped to that one vault

Result

A pipeline step can reference $(SqlConnectionString) exactly like any other variable, while the actual value only ever exists inside Key Vault and the running task's memory.

Code: Referencing Key Vault From a .NET App

- PharmacyApi's Program.cs adds the Key Vault as a configuration source — secrets resolve through IConfiguration exactly like appsettings.json values

```
var builder = WebApplication.CreateBuilder(args);

var keyVaultUri = new Uri(
    builder.Configuration["KeyVaultUri"]!);

builder.Configuration.AddAzureKeyVault(
    keyVaultUri,
    new DefaultAzureCredential());

var sqlConnectionString =
    builder.Configuration["SqlConnectionString"];

builder.Services.AddDbContext<PharmacyContext>(opt =>
    opt.UseSqlServer(sqlConnectionString));
```

Managed Identities: Overview

- An identity that Azure manages on behalf of a resource (an App Service, a Function App, a VM) — no credential for the app to store, rotate, or leak
- Azure automatically handles the underlying credential and its rotation entirely behind the scenes
- The resource authenticates to other Azure services (Key Vault, SQL, Storage) using this identity, with no connection string or client secret involved at all
- Replaces the older pattern of a service principal with a manually stored client secret — one less secret to manage, by design

Why it matters here

If an App Service's managed identity is compromised, there's no stored credential for an attacker to extract and reuse elsewhere — the identity only works from that resource.

System-Assigned vs. User-Assigned Managed Identity

System-assigned

- Created and tied directly to one resource
- Deleted automatically when that resource is deleted
- Simplest option for a single App Service
- Can't be shared across multiple resources

User-assigned

- Created as its own standalone Azure resource
- Can be assigned to multiple resources at once
- Outlives any single resource it's attached to
- Better fit when several services share one identity

Code: DefaultAzureCredential in a .NET App

- DefaultAzureCredential automatically picks the right authentication method — managed identity in Azure, the signed-in developer locally

```
// Works unchanged in every environment:
// - Local dev: uses the developer's Azure CLI login
// - Deployed: uses the App Service's managed identity
var credential = new DefaultAzureCredential();

var client = new SecretClient(
    new Uri("https://kv-pharmacyapi-dev.vault.azure.net/"),
    credential);

KeyVaultSecret secret =
    await client.GetSecretAsync("SqlConnectionString");

string connectionString = secret.Value;
```

Service Connections in Azure DevOps

- A service connection is how a pipeline authenticates to an external system — most often, an Azure Resource Manager (ARM) connection to a subscription
- It is effectively the pipeline's own identity: whatever access that connection has, every pipeline using it inherits
- Can be backed by a service principal with a stored secret, or — the preferred option — a workload identity federation setup that needs no stored credential at all
- Scoped at creation time to a subscription, a resource group, or even a single resource — the scope chosen here is the single biggest lever on blast radius

For PharmacyApi

arm-pharmacyapi-dev is scoped to rg-pharmacyapi-dev only — it cannot see, modify or delete anything in the staging or production resource groups.

Least Privilege & Scoped Access

- Least privilege means granting exactly the access a task needs, and nothing more — applied to service connections, managed identities, and human accounts alike
- Scope a service connection's role assignment to the one resource group it provisions, not the entire subscription
- Use separate service connections per environment — a compromised Dev connection should never be able to reach Production resources
- Prefer Contributor on one resource group over Owner on a subscription, every time the task doesn't genuinely require more
- Review role assignments periodically — access that made sense when a project started often outlives its justification

Ask this question

If this identity were compromised today, what's the actual blast radius? The honest answer should be “one resource group,” never “the whole subscription.”

Knowledge Check

A pipeline's service connection is granted Owner on the entire Azure subscription instead of Contributor scoped to one resource group. The Bicep template itself has no bugs. What's the actual risk?

A. Nothing — the deployment still only touches the resources named in the template

B. If that service connection is ever compromised, the blast radius is the whole subscription, not just the one resource group it manages

C. It only matters if the YAML trigger is misconfigured

D. Over-permissioning a service connection only matters for Terraform, not Bicep

Answer

B — Correct

A correct template doesn't change what an over-permissioned identity can still reach if its credentials are ever compromised. Scope is about containing the worst case, not the normal case.

- A is wrong — correctness of the template today says nothing about what a compromised credential could do tomorrow
- C is wrong — trigger configuration is unrelated to how much access a credential actually has
- D is wrong — least privilege applies equally to any Infrastructure as Code tool using that identity

PART 3

Scanning & Shift-Left Security

Shifting Security Left: The Core Idea

- “Left” refers to the pipeline timeline — code, commit, build, deploy, read left to right. Shifting left means catching a problem as early in that line as possible
- A secret caught at commit time costs one line to fix. The same secret found in production costs a credential rotation and an incident report
- This isn't a separate “security pipeline” bolted on afterward — it's gates added directly into the pipeline that's already building and deploying the application
- The goal is automated checks that run on every change, not a manual review that only happens before a major release

The core trade-off

Security doesn't have to slow delivery down when it's automated and placed early. It only slows delivery down when it's a manual gate bolted on at the very end.

Secret Scanning: Catching Leaked Credentials

- Scans every commit's diff for patterns matching known secret formats — cloud access keys, SAS tokens, private keys, connection strings
- Tools: GitHub's built-in secret scanning, open-source scanners like Gitleaks or TruffleHog as a pipeline step, or Microsoft Defender for DevOps
- Runs on every push — catching a leaked secret within minutes of it being committed, not during an occasional audit
- A critical finding fails the build immediately — block the merge, don't just leave a comment for someone to notice later

A deliberate exercise

Plant a fake API key in a branch and watch the scan catch it and fail the build — seeing the gate actually work builds far more confidence than reading about it.

Dependency Scanning: Overview

- Checks every NuGet package your application depends on against known-vulnerability databases, such as the National Vulnerability Database
- Runs as its own pipeline step, separate from compiling and testing the application itself
- Tuned to a severity threshold — fail the build only on Critical or High findings, so the gate is strict where it matters without being noisy on every minor advisory
- Catches not just the packages you reference directly, but everything they in turn depend on — a vulnerability several layers deep in the dependency tree is still your problem

Tools

dotnet list package --vulnerable (built into the .NET SDK), GitHub Dependabot, Microsoft Defender for DevOps, and commercial scanners like Mend or Snyk.

Code: Scanning NuGet Dependencies in .NET

- The .NET SDK includes a built-in command for checking NuGet packages against known vulnerabilities — easy to add as its own pipeline step

```
# Restore first so the dependency graph is resolved
dotnet restore PharmacyApi.csproj

# List only vulnerable packages, including transitive ones
dotnet list PharmacyApi.csproj package \
  --vulnerable --include-transitive

# Pipeline step: fail the build if the command
# reports any Critical or High severity finding
```

Container Scanning: Overview

- If the application ships as a container image, the image itself is an attack surface — not just the application code running inside it
- Scans the base image and every layer for known vulnerabilities — an outdated OS package or an old runtime with a published CVE the app code never touches but still ships
- Tools: Trivy, Microsoft Defender for Containers, or the registry's own built-in scanning — Azure Container Registry supports this natively
- Typically gates the push to the registry, not just the build — a vulnerable image is never allowed to become pullable in the first place

Why it's separate

Dependency scanning covers what your build tool pulled in via NuGet. Container scanning covers everything else baked into the image, including the OS layer underneath it.

Container Scanning for .NET Container Images

- A typical PharmacyApi Dockerfile and the scan step that follows it in the pipeline

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
COPY . .
RUN dotnet publish PharmacyApi.csproj -o /app

FROM base AS final
COPY --from=build /app .
ENTRYPOINT ["dotnet", "PharmacyApi.dll"]

# Pipeline step after the image build:
trivy image --severity CRITICAL,HIGH \
  --exit-code 1 pharmacyapi:$(Build.BuildId)
```

Knowledge Check

A secret-scanning step finds a hard-coded API key in a commit, but the very next commit removes that key from the file. Is it still a problem?

A. No — the current version of the file is clean, so there's nothing left to find

B. Yes — the key is still in the repository's history forever, so it must be treated as compromised and rotated

C. Only if the repository is public

D. Only if the scan is configured to check full history, not just diffs

Answer

B — Correct

Removing a line in a later commit doesn't remove it from history. The only real fix is rotating the actual credential at its source, not editing the file to hide it.

- A is wrong — the commit history still contains the key, readable by anyone who can clone the repository
- C is wrong — even a private repository has readers (teammates, pipeline identities) who can see its full history
- D is wrong — the scan already caught it in the diff at the moment it was committed, regardless of later full-history settings

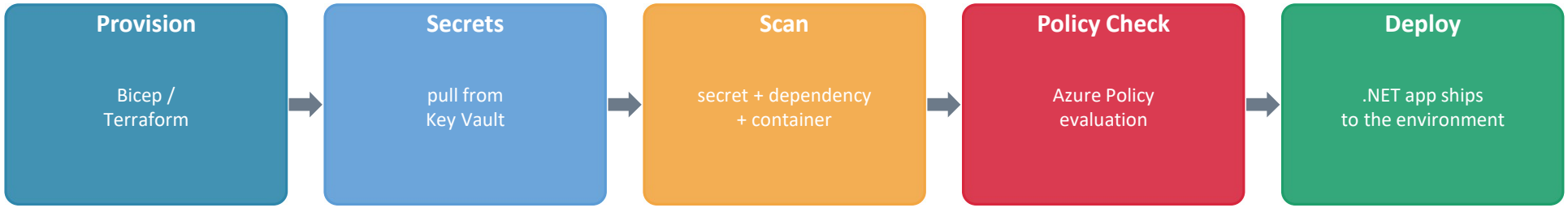
PART 4

Policy, Compliance & the Full Pipeline

Policy as Code With Azure Policy

- Azure Policy defines organizational rules as code — “deny any storage account without encryption,” “require specific tags on every resource” — enforced automatically
- A policy can run in audit mode (flag but don't block) while a team gets used to it, then switch to deny mode once it's trusted
- Policies attach to a scope — a subscription or resource group — and apply to every deployment into it, including ones coming from Bicep or Terraform
- A deployment that violates an active deny policy is rejected by Azure itself, at deployment time — an independent layer behind whatever the pipeline's own checks already caught
- Compliance reporting becomes a query against policy evaluation results, not a spreadsheet someone updates by hand once a quarter

Putting It Together: The Full Secure Pipeline



- Every stage can block the ones after it — “block, don't just report” is the principle behind every gate shown here
- The pipeline provisions its own environment, secures its own secrets, and checks its own compliance before a single line of application code is deployed

Applying This: A Hands-On Walkthrough

Provision, secure and scan — before anything deploys

- Provision (or update) an environment for PharmacyApi from a Bicep or Terraform template, run entirely by the pipeline
- Pull a secret from Azure Key Vault into a running .NET app at run time — nothing hard-coded anywhere
- Add a secret-scanning step and prove it fails the build on a planted credential
- Add a dependency-scanning step with a severity threshold and prove it fails the build on a critical NuGet finding

Full step-by-step instructions are in the separate Labs document — this slide is the "why," not the "how."

Common Pitfalls & Anti-Patterns

- Hand-editing a resource created by Bicep or Terraform directly in the Portal — the next deployment either reverts the change or fights it
- Storing Terraform state locally instead of in a shared, locked remote backend
- Granting a service connection Owner on a subscription “to save time,” then never revisiting that decision
- Treating a scan finding as advisory instead of blocking — a gate nobody has to pass isn't a gate
- Running policy in audit mode indefinitely and never promoting it to deny once it's proven reliable
- Letting Dev, Staging and Production templates quietly diverge until Staging no longer validates what Production will actually run

Best Practices Checklist

- Every environment is provisioned from a template in source control, reviewed the same way as application code
- No secret is ever hard-coded — Key Vault is the single source of truth, referenced through managed identity
- Every service connection and managed identity is scoped to exactly what it needs, per environment
- Secret, dependency and container scans run on every change and fail the build on critical findings
- Policy is enforced at the subscription or resource-group scope, not just hoped for in a document
- A plan or what-if output is reviewed before any apply that touches a shared or production environment

Key Takeaways

- Infrastructure defined in code is reviewable, diffable and repeatable — the same discipline applied to application code, now applied to what it runs on
- A secret belongs in Key Vault, never in configuration files or pipeline variables — requested at run time and never logged
- Managed identities and scoped service connections limit the blast radius of any single compromised credential
- Secret, dependency and container scanning are gates, not reports — a critical finding has to fail the build
- Azure Policy adds an independent, automatic layer of enforcement behind whatever the pipeline itself already checks
- Shifting security left means moving checks earlier where they're cheap to fix — never removing them from the end

PharmacyApi — .NET 8 Web API (C#)

Thank You

Questions & Discussion